

R Workshop – Day 1 – Getting Comfortable with R

Applied R for Statistical Teaching, Research and Reproducible Analysis

Contents

1	R Workshop – Day 1 – Getting Comfortable with R	2
1.1	Learning objectives	2
1.2	Capstone link	2
1.3	Segment plan (3 hours)	2
1.4	Part 1 – Concept bridge (Core workshop activity)	2
1.4.1	R, RStudio, scripts and the console	2
1.4.2	Why an RStudio Project	3
1.5	Part 2 – Guided coding (Core workshop activity)	3
1.5.1	2.1 Assignment, object names and comments	3
1.5.2	2.2 Vectors	3
1.5.3	2.3 Indexing and logical conditions	4
1.5.4	2.4 Missing values	4
1.5.5	2.5 A small data frame	4
1.5.6	2.6 Descriptive statistics, including grouped summaries	4
1.5.7	2.7 Reading error messages	5
1.6	Part 3 – Participant practice (Core workshop activity)	5
1.7	Optional demonstration	5
1.8	Reference or take-home material	5
1.9	Daily deliverable and the Day 1 baseline task	6
1.9.1	Baseline task (repeated on Day 7)	6
1.9.2	Evidence log entry	6
1.10	Facilitator notes	6

1 R Workshop – Day 1 – Getting Comfortable with R

Session 1 of 7 | Duration: 3 hours Programme: Applied R for Statistical Teaching, Research and Reproducible Analysis

1.1 Learning objectives

By the end of this session you will be able to:

1. Tell R, RStudio, scripts and the console apart, and know which one to use for what.
2. Start every piece of work from an RStudio Project, using relative paths instead of hard-coded folder names.
3. Create R objects with assignment, and write comments that explain *why*, not *what*.
4. Create, inspect and index vectors using positions and logical conditions.
5. Represent missing values with `NA` and use `na.rm` correctly.
6. Build a small data frame and compute descriptive statistics, including grouped summaries.
7. Read an R error or warning message and find the relevant help page.

1.2 Capstone link

Day 1 has no anchor dataset yet – that begins on Day 2. Today you complete a **baseline task** that you will repeat, with an equivalent dataset, on Day 7. The comparison between your Day 1 and Day 7 work is your own evidence of progress through the workshop.

1.3 Segment plan (3 hours)

Segment	Time	Purpose
Opening and recap	10 min	Introductions, software check, workshop map
Concept bridge	25 min	R vs RStudio, why scripts and projects matter
Guided coding	60 min	Objects, vectors, indexing, <code>NA</code> , data frames, summaries
Participant practice	45 min	Apply the same workflow to a new vector and data frame
Interpretation and reporting	25 min	Turn a summary into a sentence a colleague could read
Evidence log and capstone update	15 min	Save the baseline task and log it

1.4 Part 1 – Concept bridge (Core workshop activity)

1.4.1 R, RStudio, scripts and the console

- **R** is the language and the engine that runs your code. It exists whether or not you ever open RStudio.
- **RStudio** is an *integrated development environment* (IDE): a window manager around R that adds a script editor, a file browser, a plot viewer and a help pane.
- The **console** (bottom-left pane by default) runs one command at a time and forgets it once you close R. Use it for quick checks, not for anything you want to keep.

- A **script** (a .R file) is a saved, repeatable sequence of commands. Everything that matters for your analysis should end up in a script, not only in the console.

Working rule for this workshop: if you would be unhappy to lose it, it belongs in a script, not just in the console.

1.4.2 Why an RStudio Project

An RStudio Project (.Rproj file) pins your working directory to the project folder. This means:

- `read_csv("Data/raw/file.csv")` works for you and for anyone else who opens the same project, on any computer.
- You never need `setwd("C:/Users/yourname/Desktop/...")`, which breaks the moment the script moves to another machine.

Today's setup steps:

1. File -> New Project -> New Directory -> New Project.
2. Name it something durable, e.g. r-workshop.
3. Inside the project, create folders Data/raw, Data/processed, Scripts, Outputs (you will reuse this structure through Day 7).
4. Create a new script Scripts/day1.baseline.R and save it immediately.

1.5 Part 2 – Guided coding (Core workshop activity)

Type each block into your script and run it line by line (Ctrl/Cmd + Enter), not by pasting it all at once. Read the output before moving on.

1.5.1 2.1 Assignment, object names and comments

```
# Comments start with # and explain *why*, not *what the code obviously does*
workshop_day <- 1           # use <- for assignment, not =
participant_count <- 24
average_age <- 41.6

workshop_day
print(participant_count)
```

Object names should be readable in six months: `average_age`, not `x`, not `avgAge1`.

1.5.2 2.2 Vectors

A vector is an ordered collection of values of the *same* type.

```
ages <- c(34, 41, 29, 52, 38, NA, 45)
class(ages)
length(ages)

regions <- c("North West", "South East", "South West", "North Central")
class(regions)
```

1.5.3 2.3 Indexing and logical conditions

```
ages[1]           # first element
ages[c(1, 3)]     # first and third
ages[ages > 40]    # logical indexing: keep ages above 40

over_40 <- ages > 40 # this is itself a vector of TRUE/FALSE/NA
over_40
```

Logical indexing is the single most useful R skill for the rest of this workshop: almost every data-cleaning rule you write from Day 2 onward is a logical condition like this one.

1.5.4 2.4 Missing values

```
mean(ages)          # returns NA -- R refuses to silently guess
mean(ages, na.rm = TRUE)
sum(is.na(ages))     # how many missing values?
which(is.na(ages))   # which positions?
```

NA means “unknown”, not zero and not “missing on purpose”. R propagates NA through almost every calculation until you tell it otherwise with `na.rm = TRUE`. This is a safety feature, not an inconvenience – it stops you reporting a mean you did not actually compute.

1.5.5 2.5 A small data frame

```
participants <- data.frame(
  participant_id = c("P01", "P02", "P03", "P04", "P05"),
  age = c(34, 41, 29, 52, 38),
  region = c("North West", "South East", "South West", "North Central", "South
    ↪ East"),
  pre_score = c(58, 64, 71, 49, 60)
)

str(participants)
head(participants, 3)
nrow(participants)
ncol(participants)
```

1.5.6 2.6 Descriptive statistics, including grouped summaries

```
mean(participants$pre_score)
median(participants$pre_score)
sd(participants$pre_score)

# Grouped summary without extra packages
tapply(participants$pre_score, participants$region, mean)
```

We will redo grouped summaries with `dplyr::group_by()` and `summarise()` from Day 2 onward, because it scales better to many groups and many statistics at once. `tapply()` is shown here so the underlying idea – split, compute, combine – is clear before the package hides the mechanics.

1.5.7 2.7 Reading error messages

Run this deliberately:

```
mean(participnts$pre_score)
```

R replies with something like `Error in mean(participnts$pre_score) : object 'participnts' not found`. Read right to left: *what* went wrong (object not found) and *where* (inside `mean(...)`). Most beginner errors are typos, mismatched brackets/quotes, or using `=` where `==` was meant. When stuck, run `?function_name` (e.g. `?mean`) to open the help page, or `example(function_name)` for runnable examples.

1.6 Part 3 – Participant practice (Core workshop activity)

Work in your script. Do not look at the solution until you have tried each step.

1. Create a vector `attendance` with 8 values between 0 and 100, including one NA.
2. Compute the mean attendance, ignoring the NA.
3. Create a logical vector flagging attendance below 70.
4. Build a data frame with columns `student_id` (made up), `attendance`, and `pass` (TRUE/FALSE for `attendance >= 70`, using your logical vector or an `ifelse()`).
5. Compute the proportion of students who pass.
6. Deliberately introduce one typo and read the resulting error message out loud to your neighbour before fixing it.

A facilitator-reviewed solution is available in `Solution-Scripts/day1_practice_solution.R` – check it only after attempting the task yourself.

1.7 Optional demonstration

These are useful if the group is moving quickly. They are not required for the daily deliverable.

- **Matrices and basic matrix operations** – `matrix()`, `t()`, `%*%`. These return in full on Day 6 when you build a design matrix by hand.
- **Factors** – `factor()` for ordered or fixed categories, and why factors behave differently from plain character vectors in summaries and models.
- **Lists and `typeof()`** – a list can hold mixed types and different lengths; `typeof()` and `class()` distinguish the underlying storage from the object's behaviour.

```
# Optional: factors
satisfaction <- factor(c("Low", "High", "Medium", "High"),
                      levels = c("Low", "Medium", "High"), ordered = TRUE)
satisfaction
satisfaction < "High"
```

1.8 Reference or take-home material

- Extended debugging examples (mismatched parentheses, dangling commas, case-sensitive names).
 - More practice with subsetting: `[], [[, $`, and when each is required.
 - RStudio keyboard shortcuts: `Ctrl/Cmd + Enter` (run line), `Ctrl/Cmd + Shift + M` (pipe `|>`), `Ctrl/Cmd + 1/2` (move between script and console).
-

1.9 Daily deliverable and the Day 1 baseline task

Deliverable: a working RStudio project containing `Scripts/day1_baseline.R`, your practice exercise, and the baseline task below, all saved inside the project folder.

1.9.1 Baseline task (repeated on Day 7)

A small dataset is provided at `Data/raw/day1_baseline_sample.csv` (12 students, two columns of interest plus an ID). It contains at least one missing value and at least one invalid value – you do not need to find every issue today, just demonstrate the workflow.

1. Import the dataset with `read.csv()` (we introduce `readr::read_csv()` properly on Day 2).
2. Identify missing or clearly invalid values (for example, a negative score or an attendance percentage above 100) using the indexing skills from Part 2.
3. Produce one numerical summary (e.g. mean test score, ignoring missing/invalid values) and one labelled plot (a basic `hist()` or `plot()` is enough today – `ggplot2` starts Day 3).
4. Write three sentences interpreting the result for a non-technical reader (a department head, not a statistician).
5. Save the script, the plot, and your three sentences inside the project folder, e.g. `Outputs/day1_baseline_summary`

```
baseline <- read.csv("Data/raw/day1_baseline_sample.csv")
str(baseline)
summary(baseline$test_score)
hist(baseline$test_score, main = "Day 1 baseline: test scores", xlab = "Score")
```

1.9.2 Evidence log entry

Open `Workshop-Evidence-Log.md` and complete the Day 1 row: note the baseline task is saved, and that your R/RStudio setup is confirmed working.

1.10 Facilitator notes

- Confirm every participant has R $\geq 4.x$ and a recent RStudio Desktop installed *before* this session; software installation should not consume live-session time.
- The Day 1 baseline task is intentionally small. Its only purpose is to give Day 7 something to compare against – do not let it expand into a full analysis.
- Keep “Optional demonstration” genuinely optional: if the group needs the full 60 minutes of guided coding plus 45 minutes of practice, skip the optional section entirely.